GORDON GROUP MEETING

Terco

GPU integrals in standard Fortran: the avenue for GAMESS on GPUs

Jorge Luis Gálvez Vallejo
jorge.galvezvallejo@anu.edu.au

NCI · ANU
Australian National University
Canberra, Australia / October 14, 2026



1 Crawl, walk, run

LibERI and GAMESS OpenMP crawled, gmshpc walked, so that Terco could run

Crawled: LibERI, GAMESS omp offload

GPU ERIs in GAMESS, then OpenMP target and `do concurrent`.

Alkan et al., JCP 161, 082501; del Angel et al., f functions, JCP 164, 214118

One shell quartet at a time starves the GPU.

Walked: gmshpc

libcchem in CUDA C with streams, byproduct of the ECP.

Binned shell pairs keep the GPU busy, but it is a second code base with a CUDA maintenance burden. Also half the group does not speak C++.

Runs: Terco

Is the difference algorithmic or the Fortran/CUDA gap?

Explore the limits of Fortran standard parallelism.

2 How Terco works

Standard Fortran only (do concurrent)

Kernels are pure `do concurrent` (`nvfortran -stdpar=gpu`). OpenACC only moves data and does atomics. The same source builds with gfortran on the CPU, and CI converts it to OpenMP.

Binned shell pairs (ECP/libcchem/gmshpc)

Quartets are never stored. Shell pairs are binned by type (l_a, l_b) and Schwarz exponent; a bin pair is kept if $s_{ab} + s_{cd} \leq -\log_{10} \tau$. Each thread derives its quartet from its index.

Integrals per class

HGP: VRR with Gill's sieve, contraction, then HRR, 81 classes through d. Direct 8-fold digestion into J and K, scaled for hybrids. Rys per class covers f.

Basis and data

The caller supplies the basis (BSE JSON or plain arrays). Kernels run in Cartesians, spherical on top. Data stays on the device for the whole SCF.

Why bins

At 75 waters/6-31G the stored quartet list cost 26 s of host time and 11.7 GB. Binning by type also sorts by class for free.

2 Binning shell pairs

Pairs are bucketed once; bin pairs are admitted by an integer test on the host

shell pair ab

$$Q_{ab} = \sqrt{\max|(ab|ab)|}$$

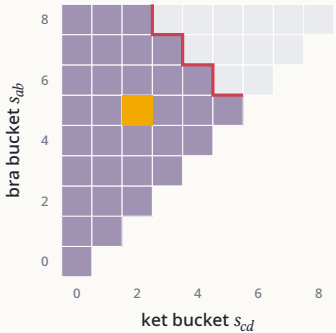
$$s_{ab} = \mathrm{int}(-\log_{10} Q_{ab})$$

bin key $(l_a, l_b, K_{ab}, s_{ab})$

type, contraction degree, Schwarz decade: every pair in a bin shares all four

keep (A, B) iff $s_A + s_B \le -\log_{10}\tau$

$\tau = 10^{-10},\ B \le A$



segments, sorted by class

(pp|ss) $\|A\|\|B\|$ quartets
(pp|ss) $\|A\|\|B\|$ quartets
(pp|ss) $\|A\|(\|A\|+1)/2$ (diag)
…

do `concurrent` over t
one launch per class

$seg = \mathrm{bsearch}(sOff, t)$
$(ab, cd) \leftarrow t - sOff(seg)$

Quartets are never stored: a thread derives its own (ab, cd) from its index. Conservative: never admits less than τ asks.

2 The Fock build as an algorithm

Host plans once per build; the device does everything per quartet

Algorithm 1: Plan (host)

Input: bins, τ, $D_{\max}$ per bin

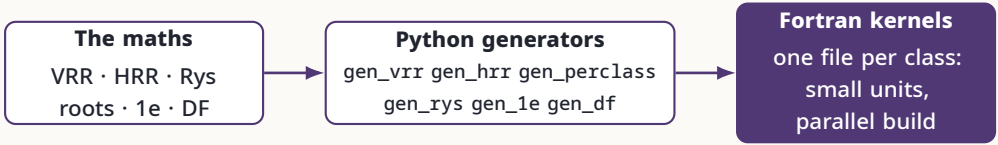
$S \leftarrow \mathrm{int}(-\log_{10}\tau)$;
foreach *bin pair* (A, B), $B \le A$ **do**
  if $s_A + s_B > S$ **then skip**;
  if *density bound of* $(A, B) < \tau$ **then skip**;
  append segment; $n \leftarrow |A||B|$ (or $|A|(|A|+1)/2$);
  sOff $\leftarrow$ *sOff* $+ n$; *// prefix sum*
sort segments by class $(l_a l_b | l_c l_d)$;
foreach *class* **do**
  launch its kernel (HGP or Rys);

Algorithm 2: Generated class kernel (device)

do concurrent *t in this class's work*
  $seg \leftarrow \mathrm{bsearch}(sOff, t)$;
  $(ab, cd) \leftarrow \mathrm{decode}(t - sOff_{seg})$;
  if $Q_{ab} Q_{cd}\, D_{\max} < \tau$ **then cycle**;
  $(ab|cd) \leftarrow$ VRR, contract, HRR ; *// or Rys*
  digest 8-fold into J, K ; *// atomics*

Every thread in a launch shares (l_a, l_b, l_c, l_d), so ptxas budgets registers for one class, not the worst one. No launch geometry is given: `do concurrent` leaves it to the compiler.

2 The kernels are generated, not hand-written



Never edited by hand

A new algorithm is a generator change. CI regenerates the committed kernels and requires them byte for byte.

Rys from the HGP skeleton

Shared decode, screening and digestion; only the integral body differs. Under 1 s to generate, where full HGP text at f took 6 min and 136 MB.

DFT the same way

ExchCXX functionals machine-translated to Fortran: XC on the device, no libxc. **32 run today, wB97X-V included**; the rest say what they need.

3 Performance through d, and f now works

6.8×

faster than LibERI

w404, 6-31G (5k basis functions); one Fock build: 22 s vs 149 s

±10%

of gmshpc's CUDA C

w172, 1e-10: 6-31G* 2.26 vs 2.37 s; 6-31G 0.968 vs 0.887 s

0.92×

of GPU4PySCF's time (d)

w20, cc-pVTZ without f (1100 AOs), Fock build: 0.77 vs 0.84 s

Milestone: f functions

- Every class with an f shell is one of **175 generated Rys kernels**; HGP stays at d.
- Matches pyscf on the first build: Fock 8e-13 (water/cc-pVTZ), 2.6e-11 (formamide/def2-TZVP); RHF to 6e-12.
- Roots to 9, so g is reachable. Full GPU build ∼32 min on 4 cores.

cc-pVTZ Fock, 1e-13	Terco	GPU4PySCF
w10, f (650 AOs)	0.70 s	0.65 s
w20, f (1300 AOs)	2.20 s	1.44 s
w20, no f (1100)	0.77 s	0.84 s

The f kernels are untuned; tuning has not started.

One V100. Two-electron gradient against GPU4PySCF, 6-31G*: 0.51× its time at w20, 1.24× at w150 (8.41 vs 6.8 s).

4 New: an analytic two-electron gradient

Rys, derivatives by raising angular momentum, D by translational invariance, RHF and UKS-ready, in pure do `concurrent`. w150 6-31G*, 1e-13:

31.8 s → 8.4 s

1. **Ordered compaction:** screen, then compute only survivors; ket-uniform warps 6% → 68%
2. **Lane-private gradients:** no same-address FP64 atomics; (ss|ss) −47%
3. **Chunked heavy classes:** (pp|pp) 0.64 → 0.13 s, (dp|pp) 1.56 → 0.33 s

Validated, not frozen

pyscf `int2e_ip1` to 1e-12 and finite differences; no test checks Terco against its own output. A CUDA twin of the same algorithm was within 1.1–1.3×: the gap left is algorithmic.

Running today, and next

DFT through wB97X-V (range separation, VV10), RKS and UKS, pyscf to ∼3e-11; f energies; MPI and a `bind(c)` ABI.

Next: f tuning, f gradients, meta-GGAs, density fitting.

For GAMESS

GPU HF, DFT and gradients in standard Fortran its developers can read and extend: no CUDA, no C++.

4 Gradient: ordered survivor compaction

Screen first, then compute only survivors, in an order that keeps warps uniform

Fused: one warp, 32 work items



every lane screens, then the survivors run the integral body while the rest idle: divergence

Screen pass: chunks of 32 (one warp), one atomic capture per chunk



Compact pass: dense survivor list, in item order



decode is bra-fastest, so neighbours share a ket pair; the next chunk's survivors fill the warp

Lane-private gradients



survivor i adds into copy $\mathrm{mod}(i-1, 32)+1$: no same-address FP64 atomics; summed once at the end

Measured, w150 6-31G*

(ps|ss): fused 3.97 s → per-item append 3.31 s → chunks of 32 2.15 s

(ss|ss)+(ps|ss): 1 copy 3.11 s → 32 copies 1.91 s

Survivor pattern schematic. Both passes are sliced from the same generated text: nothing in the integral body is retyped.

Thank you

GPU HF, DFT and gradients for GAMESS, from a standard-Fortran library its developers can read and extend.

github.com/JorgeG94/terco



Jorge Luis Gálvez Vallejo

jorge.galvezvallejo@anu.edu.au